```
#Import Libraries and Load Dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


df = pd.read_csv("marketing_data.csv")



df.head()

# Shape of dataset
print(df.shape)

# Column names
print(df.columns)

# Dataset information
df.info()

# Check missing values
df.isnull().sum()
```

```
# Convert Dt_Customer into datetime format
df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'])

# Verify datatype
print(df['Dt_Customer'].dtype)
```

```
# Unique values in Education
print(df['Education'].unique())

# Unique values in Marital_Status
print(df['Marital_Status'].unique())
```

```
['Graduation' 'PhD' '2n Cycle' 'Master' 'Basic']
['Divorced' 'Single' 'Married' 'Together' 'Widow' 'YOLO' 'Alone' 'Absurd']
```

```
# Replace unusual categories
df['Marital_Status'] = df['Marital_Status'].replace({
    'Alone': 'Single',
    'YOLO': 'Single',
    'Absurd': 'Single',
    'Together': 'Married'
})

# Check updated categories
print(df['Marital_Status'].unique())
```

```
# Impute missing income values

# Clean column names to remove any leading/trailing whitespace
df.columns = df.columns.str.strip()
```

```
# Ensure 'Income' column is numeric by cleaning it and converting to float
# This step must happen before calculating the mean for imputation.
df['Income'] = df['Income'].replace({r'\$': '', ',': ''}, regex=True).astype(float)

# Now, impute missing income values
df['Income'] = df['Income'].fillna(
    df.groupby(['Education', 'Marital_Status'])['Income']
    .transform('mean')
)

# Check remaining missing values
print(df['Income'].isnull().sum())
```

```
0
```

```
# Create total children column
df['Total_Children'] = df['Kidhome'] + df['Teenhome']

df[['Kidhome', 'Teenhome', 'Total_Children']].head()
```

	Kidhome	Teenhome	Total_Children
0	0	0	0
1	0	0	0
2	0	1	1
3	1	1	2
4	1	0	1

```
# Current year
current_year = 2026

# Create age column
df['Age'] = current_year - df['Year_Birth']

df[['Year_Birth', 'Age']].head()
```

	Year_Birth	Age
0	1970	56
1	1961	65
2	1958	68
3	1967	59
4	1989	37

```
# Create total spending column
df['Total_Spending'] = (
    df['MntWines'] +
    df['MntFruits'] +
    df['MntMeatProducts'] +
    df['MntFishProducts'] +
    df['MntSweetProducts'] +
    df['MntGoldProds']
)

df[['Total_Spending']].head()
```

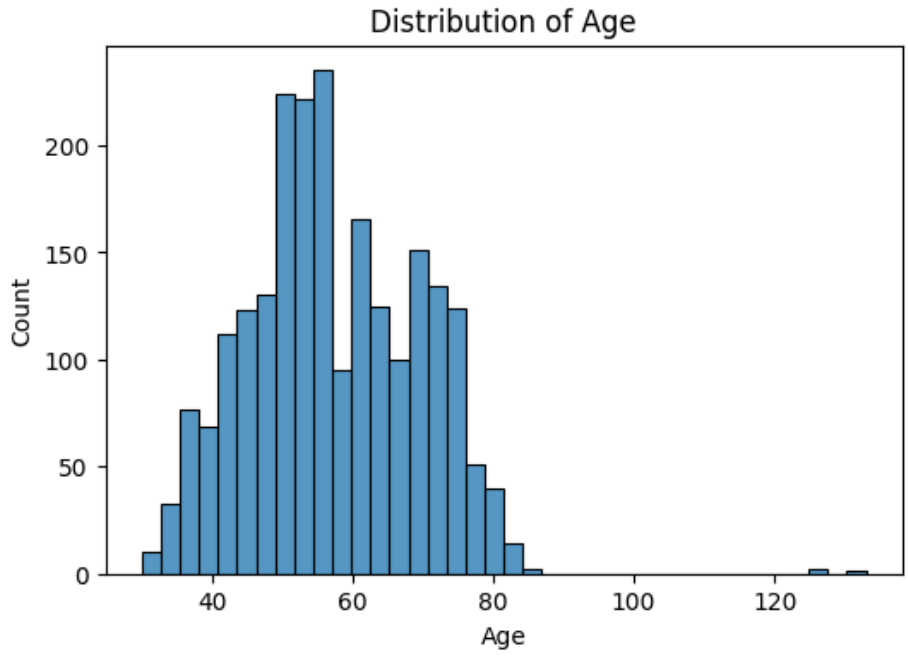
	Total_Spending
0	1190
1	577
2	251
3	11
4	91

```
# Total purchases across all channels
df['Total_Purchases'] = (
    df['NumWebPurchases'] +
    df['NumCatalogPurchases'] +
    df['NumStorePurchases']
)

df[['Total_Purchases']].head()
```

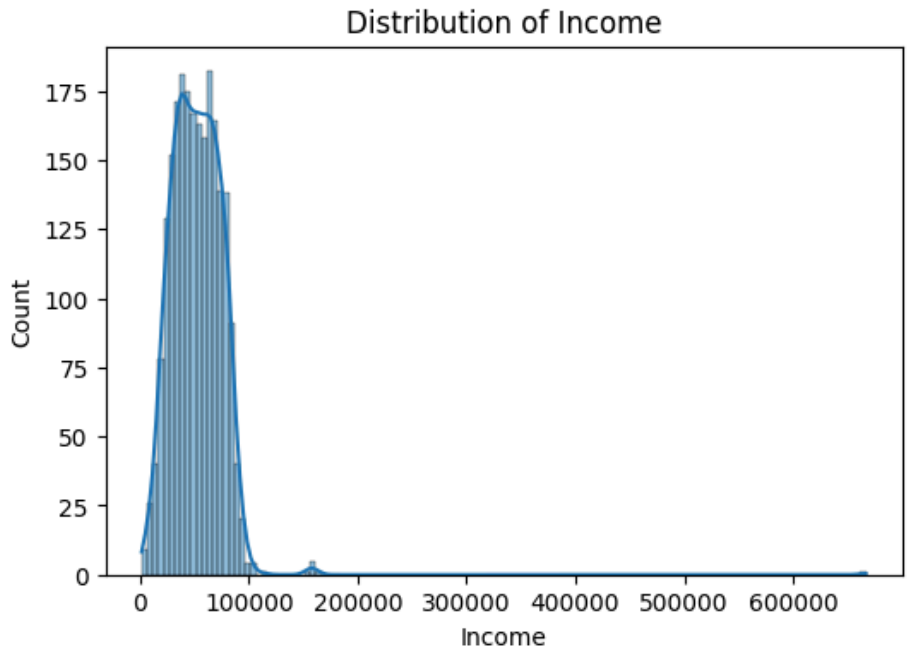
	Total_Purchases
0	14
1	17
2	10
3	3
4	6

```
# Histogram for Age
plt.figure(figsize=(6,4))
sns.histplot(df['Age'])
plt.title("Distribution of Age")
plt.show()
```

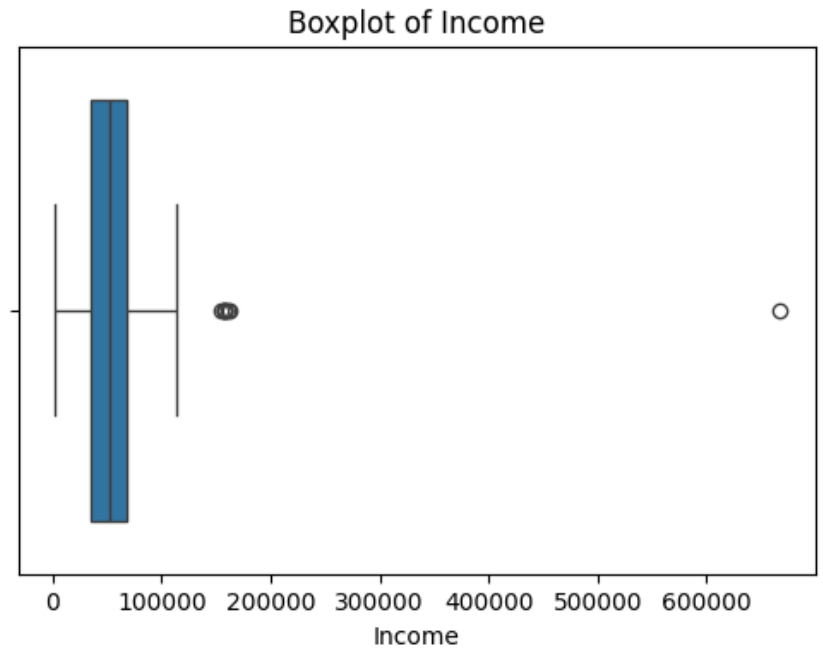



```
# Histogram for Income
plt.figure(figsize=(6,4))
sns.histplot(df['Income'], kde=True)
```

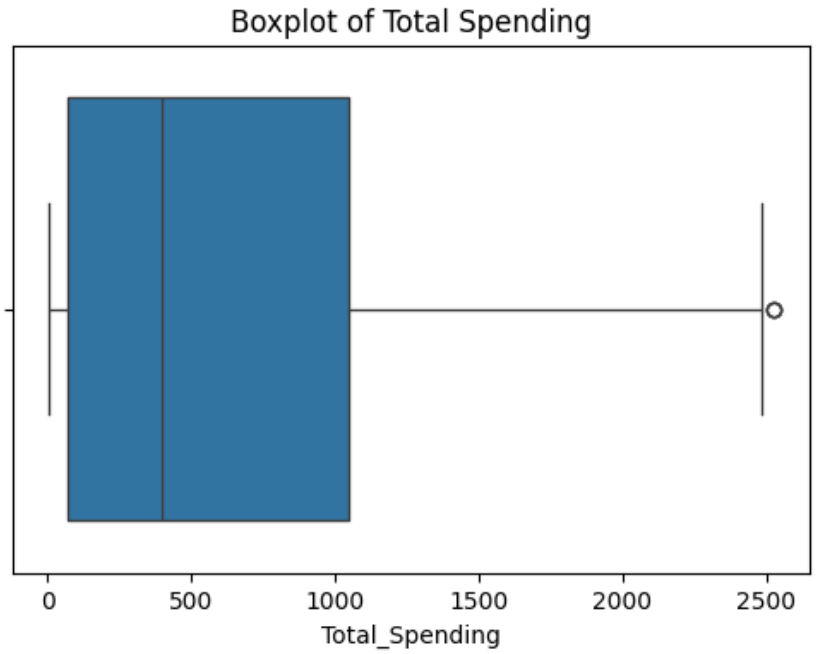
```
plt.title("Distribution of Income")
plt.show()
```




```
# Boxplot for Income
plt.figure(figsize=(6,4))
sns.boxplot(x=df['Income'])
plt.title("Boxplot of Income")
plt.show()
```




```
# Boxplot for Total Spending
plt.figure(figsize=(6,4))
sns.boxplot(x=df['Total_Spending'])
plt.title("Boxplot of Total Spending")
plt.show()
```




```
# Calculate Q1 and Q3
Q1 = df['Income'].quantile(0.25)
Q3 = df['Income'].quantile(0.75)

# IQR
IQR = Q3 - Q1

# Lower and Upper limits
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Remove outliers
df = df[(df['Income'] >= lower) & (df['Income'] <= upper)]

print(df.shape)
```

```
(2232, 32)
```

```
education_order = {
    'Basic': 1,
    'Graduation': 2,
    'Master': 3,
    'PhD': 4
}

df['Education_Encoded'] = df['Education'].map(education_order)

df[['Education', 'Education_Encoded']].head()
```

```
# One-hot encoding
df = pd.get_dummies(df, columns=['Marital_Status'], drop_first=True)

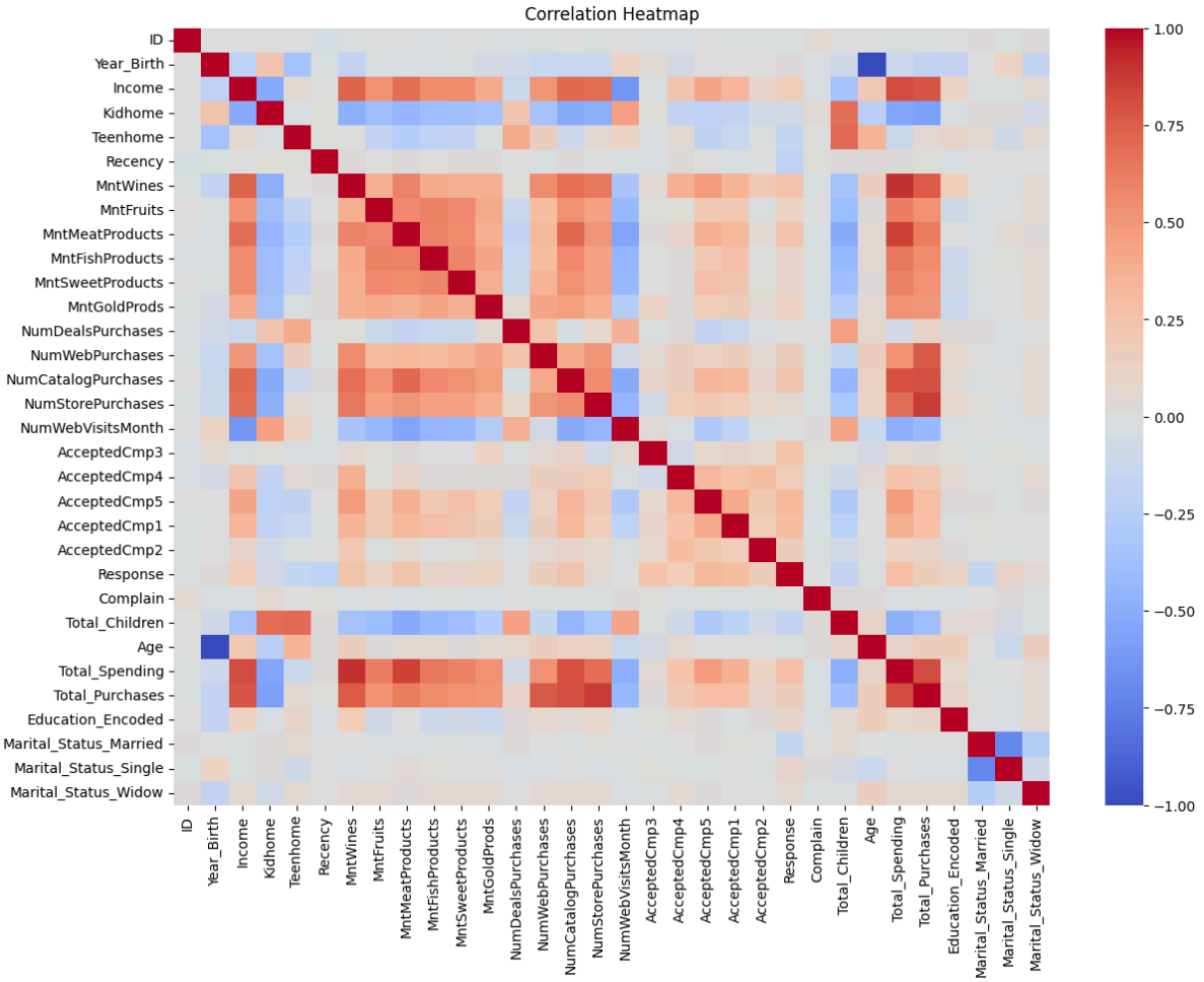
df.head()
```

	ID	Year_Birth	Education	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	M
0	1826	1970	Graduation	84835.0	0	0	2014-06-16	0	189	
1	1	1961	Graduation	57091.0	0	0	2014-06-15	0	464	
2	10476	1958	Graduation	67267.0	0	1	2014-05-13	0	134	
3	1386	1967	Graduation	32474.0	1	1	2014-05-11	0	10	
4	5371	1989	Graduation	21474.0	1	0	2014-04-08	0	6	

5 rows × 35 columns

```
# Correlation matrix
corr = df.corr(numeric_only=True)

# Plot heatmap
plt.figure(figsize=(14,10))
sns.heatmap(corr, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```
# Correlation between Age and Store Purchases
correlation = df['Age'].corr(df['NumStorePurchases'])

print("Correlation:", correlation)
```

```
Correlation: 0.12757699136193568
```

```
# Correlation between children and web purchases
correlation = df['Total_Children'].corr(df['NumWebPurchases'])

print("Correlation:", correlation)
```

```
Correlation: -0.14944393634857053
```

```
# Correlation between Store and Web purchases
correlation = df['NumStorePurchases'].corr(df['NumWebPurchases'])

print("Correlation:", correlation)
```

```
Correlation: 0.4990298499362481
```

```
# Total purchases by country
country_sales = df.groupby('Country')['Total_Purchases'].sum()

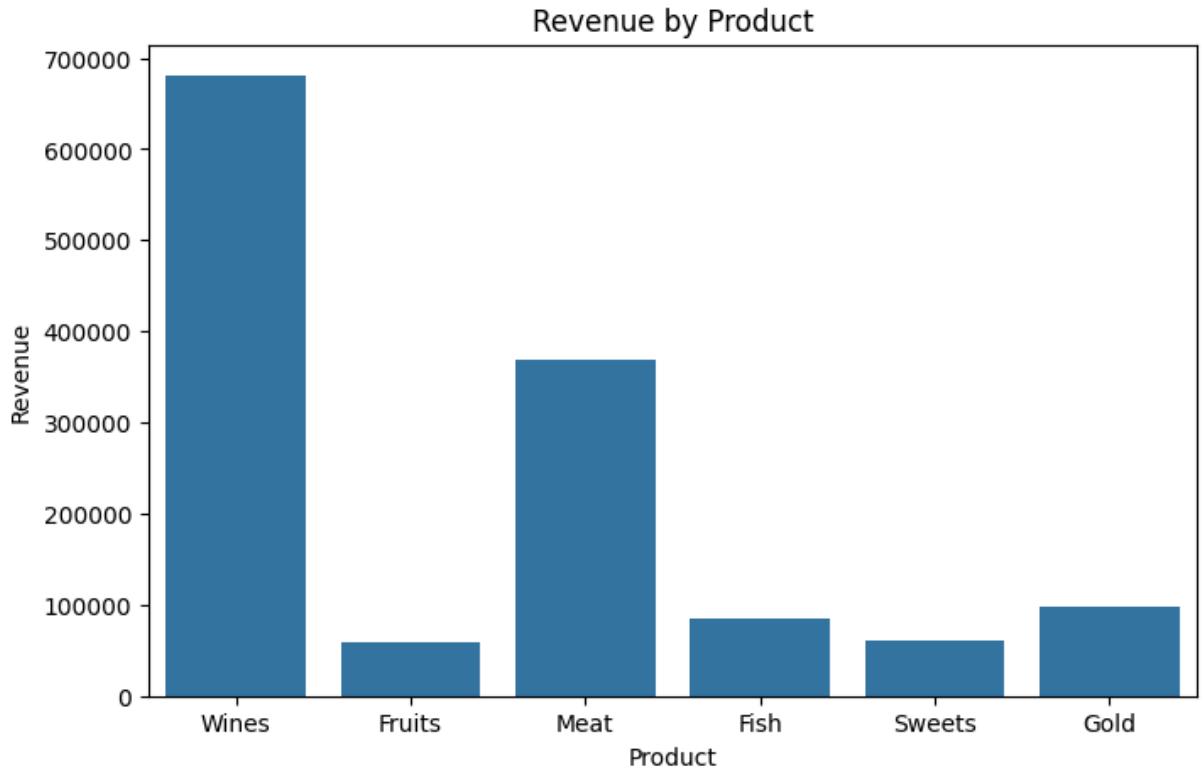
print(country_sales.sort_values(ascending=False))
```

```
Country
SP     13580
SA      4267
CA      3457
AUS     1952
IND     1712
GER     1530
US      1444
ME        52
Name: Total_Purchases, dtype: int64
```

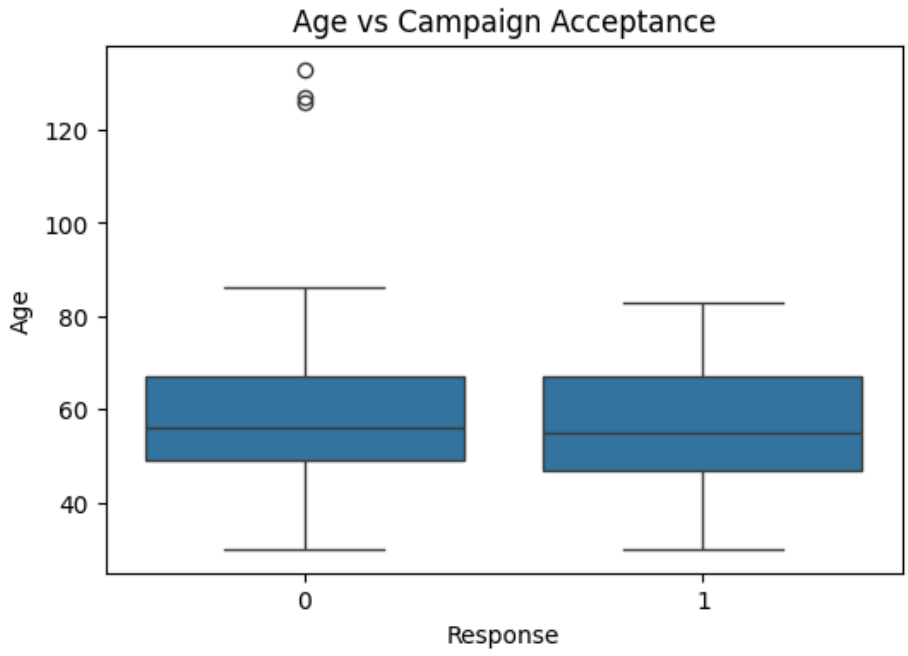
```
products = {
    'Wines': df['MntWines'].sum(),
    'Fruits': df['MntFruits'].sum(),
    'Meat': df['MntMeatProducts'].sum(),
    'Fish': df['MntFishProducts'].sum(),
    'Sweets': df['MntSweetProducts'].sum(),
    'Gold': df['MntGoldProds'].sum()
}

product_df = pd.DataFrame(products.items(), columns=['Product', 'Revenue'])

# Bar plot
plt.figure(figsize=(8,5))
sns.barplot(x='Product', y='Revenue', data=product_df)
plt.title("Revenue by Product")
plt.show()
```




```
plt.figure(figsize=(6,4))
sns.boxplot(x='Response', y='Age', data=df)
plt.title("Age vs Campaign Acceptance")
plt.show()
```



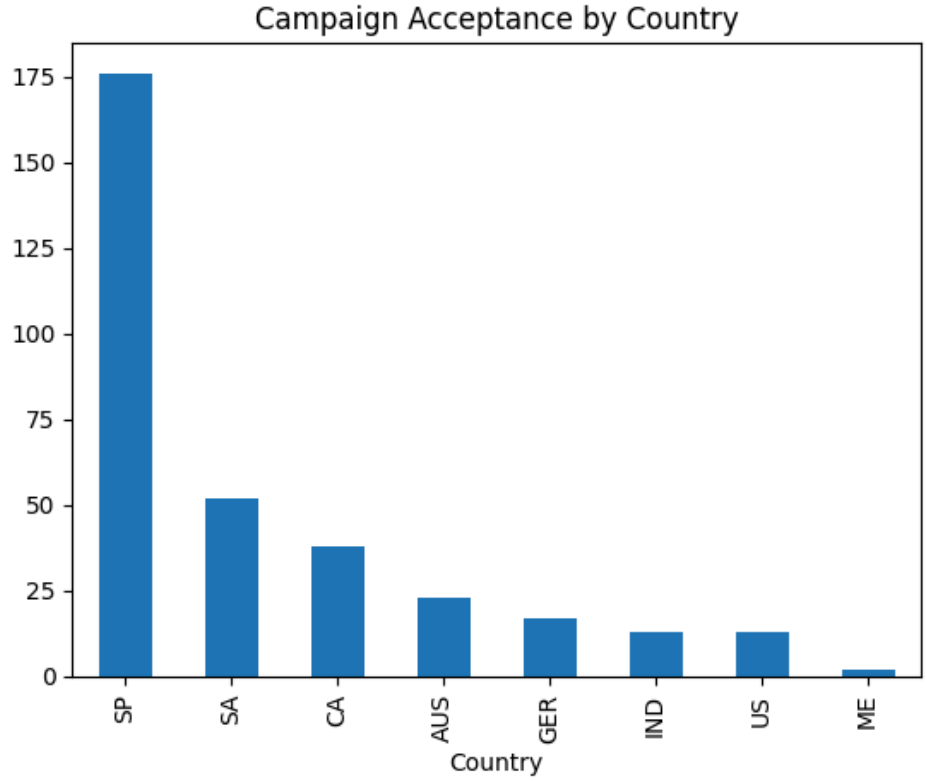
```
accepted = df[df['Response'] == 1]

country_acceptance = accepted['Country'].value_counts()

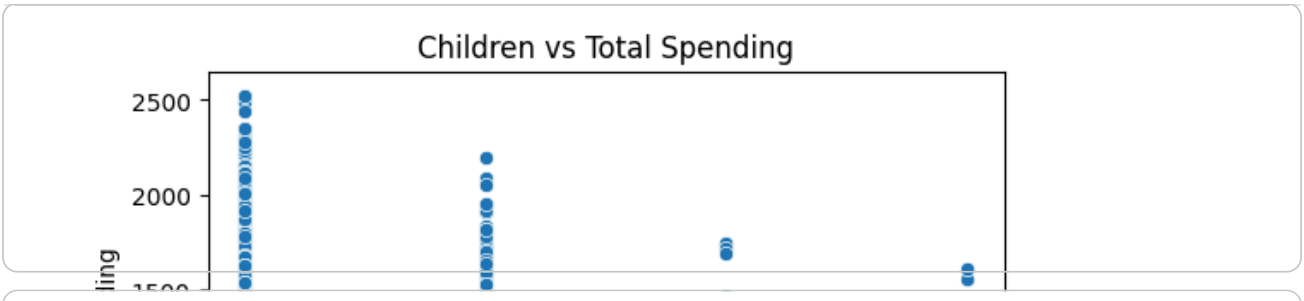
print(country_acceptance)

# Plot
country_acceptance.plot(kind='bar')
plt.title("Campaign Acceptance by Country")
plt.show()
```

```
Country
SP     176
SA      52
CA      38
AUS     23
GER     17
IND     13
US      13
ME       2
Name: count, dtype: int64
```




```
plt.figure(figsize=(6,4))
sns.scatterplot(x='Total_Children', y='Total_Spending', data=df)
plt.title("Children vs Total Spending")
plt.show()
```




```
complaints = df[df['Complain'] == 1]

education_counts = complaints['Education'].value_counts()

print(education_counts)

# Plot
education_counts.plot(kind='bar')
plt.title("Education of Customers with Complaints")
plt.show()
```

Total_Children

```
Education
Graduation    14
2n Cycle       4
Master         2
PhD            1
Name: count, dtype: int64
```

